Optimizer

Reference Manual

Table of Contents

WHAT IS THE CATALINA OPTIMIZER? 3
Status 3
Features 3
License 3
USING THE OPTIMIZER 6
USING THE OPTIMIZER WITH BLACKBOX AND BLACKCAT 6
USING THE OPTIMIZER WITH CODE::BLOCKS 7
OPTIMIZER OPTIONS 7
OPTIMIZER EXAMPLE 8
OPTIMIZER PHASES 10
TYPICAL OPTIMIZER RESULTS 12
OPTIMIZING HAND-CODED LMM PASM 13
THE CURRENT OPTIMIZER RELEASE 14
What's new in this release? 14
What's due in the next release? 14
Reporting Bugs 14

What is the Catalina Optimizer?

The Catalina Optimizer is a code optimizer that can substantially reduce the code size and increase the performance of programs compiled using the Catalina C compiler.

The Catalina C compiler is a free, open source, ANSI compliance C compiler for the Parallax Propeller. It can be downloaded from http://catalina-c.sourceforge.net/

Status

The Catalina Optimizer is released with each version of Catalina. You should only use the Optimizer that coincides with the same Catalina release. For a list of differences since the previous release, see the section later in this document titled What's new in this release?

Features

- Seamless integration with the Catalina C compiler;
- Five optimization levels;
- Linux and Windows versions;
- Optimizes C user programs as well as C library code, and also compatible LMM PASM functions;
- Works with all Catalina memory models (LMM, XMM, CMM, NMM);
- Typically reduces program code size by 10 to 15%;
- Typically improves program performance by 5% to 15%;
- Support for both Propeller 1 and Propeller 2 programs.

License

Unlike the Catalina C compiler, the Catalina Optimizer is **free,** but is it **not** open source. Only binary versions of the Optimizer are provided.

The following license applies to the Catalina Optimizer:

```
SINGLE USER SOURCE CODE LICENSE AGREEMENT

IMPORTANT - READ CAREFULLY

The license(s) granted herein are being granted pursuant to the
purchase by you (the "Licensee") of certain software products to
be used by Licensee, and, if applicable, by employees,
subcontractors or other agents of the Licensee in performing
services for Licensee on Licensee premises (such employees,
subcontractors and agents collectively referred to herein as
"Employees").

By opening and installing the software package, or exercising your
rights to make and use copies of the Software (as may be provided
for below), the Licensee agrees to be bound by, and to require
each of its Employees to be bound by, this License Agreement (the
"Agreement"). If you do not agree to the terms of this Agreement,
then DO NOT install or use the software product. If you would like
```

specific rights not granted in this Agreement, please contact Ross Higson (ross@thevastydeep.com) for more information.

SOFTWARE PRODUCT LICENSE

The Licensee and each individual Employee who views, links to, or in any way utilizes (via electronic media or otherwise) the Software (i.e. each User), must have his or her own license to use the Software and may only use the Software on a single computer (typically one personal computer) in accordance with the terms and conditions set forth herein.

OWNERSHIP RIGHTS

The enclosed software, including, but not limited to, one or more of the following: source code, object code, dynamic link libraries, static libraries, header files, sample programs, utility programs, make files and scripts, together with the accompanying documentation (collectively known as the "Software") is owned by Ross Higson and is protected by copyright laws and international treaties. Therefore, you must treat the Software like any copyrighted material (e.g., a book or musical recording) except that you may make a reasonable number of copies of the Software solely for backup purposes. However, you may make an unlimited number of copies (either in hardcopy or electronic form) of any Documentation included with the software, provided that such copies shall be used solely for internal purposes and shall not be republished or distributed to any third party.

NOT WITH STANDING ANY PROVISIONS IN THIS AGREEMENT TO THE CONTRARY, YOU MAY NOT

1. Use, copy, modify, merge or compile all or any portion of the source code, object code or executable form of the Software except as expressly provided in this agreement.

2. Make telecommunication transmittals of the Software.

3. Distribute any portion of the Software or any derivative of any portion of the Software in a software development product or otherwise in competition with Ross Higson's distribution of the Software;

4. Decompile, disassemble or reverse engineer any object code or executable form of any portion of the Software;

5. Rent or lease the Software;

6. Disclose any source code of the Software to any person or entity;

THE SOFTWARE AND ACCOMPANYING WRITTEN MATERIALS (INCLUDING INSTRUCTIONS FOR USE) ARE PROVIDED "AS IS". ROSS HIGSON DOES NOT WARRANT, GUARANTEE OR MAKE ANY REPRESENTATIONS REGARDING THE USE, OR THE RESULTS OF USE, OF THE SOFTWARE OR WRITTEN MATERIALS IN TERMS OF CORRECTNESS, ACCURACY, RELIABILITY, CURRENT, OR OTHERWISE. YOU ASSUME THE ENTIRE RISK AS TO THE RESULTS AND PERFORMANCE OF THE SOFTWARE.

Ross Higson does not warrant that the Software will meet your requirements, that operation of the Software will be uninterrupted or error-free, or that all Software errors will be corrected. Ross Higson is not responsible for problems caused by changes in the operating characteristics of computer hardware or computer operating systems which are made after the installation of the

Software nor for problems in the interaction of the Software with any other software. Ross Higson will have no responsibility to replace or refund the license fee for media damaged by accident, abuse or misapplication.

THIS SOFTWARE IS INTENDED FOR COMMERCIAL, INSTITUTIONAL, AND INDUSTRIAL USERS. THE ABOVE WARRANTIES ARE EXCLUSIVE AND IN LIEU OF ALL OTHER WARRANTIES, WHETHER EXPRESSED OR IMPLIED, INCLUDING THE IMPLIED WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NON-INFRINGEMENT. NO ORAL OR WRITTEN INFORMATION OR ADVICE GIVEN BY ROSS HIGSON, OR ANY DISTRIBUTORS, DEALERS OR AGENTS SHALL INCREASE THE SCOPE OF THE ABOVE WARRANTIES OR CREATE ANY NEW WARRANTIES. NEITHER ROSS HIGSON NOR ANYONE ELSE WHO HAS BEEN INVOLVED IN THE CREATION, PRODUCTION OR DELIVERY OF THE SOFTWARE SHALL BE LIABLE FOR ANY DIRECT, INDIRECT, CONSEQUENTIAL OR INCIDENTAL DAMAGES (INCLUDING DAMAGES FOR LOSS OF BUSINESS PROFITS, BUSINESS LOSSES, BUSINESS INTERRUPTION, LOSS OF DATA, AND THE LIKE) ARISING OUT OF THE USE OR INABILITY TO USE THE SOFTWARE, OR ANY OTHER CLAIM BY ANY PARTY EVEN IF ROSS HIGSON HAS BEEN ADVISED OF THE POSSIBILITY OF SUCH DAMAGES. IN THE EVENT ROSS HIGSON IS LIABLE FOR ANY DAMAGES, ROSS HIGSON'S LIABILITY FOR DAMAGES SHALL BE LIMITED TO THE AMOUNT PAID FOR THE SOFTWARE UNDER THE TERMS AND CONDITIONS OF THIS AGREEMENT.

Using the Optimizer

The Catalina Optimizer is not normally invoked independently – it is normally invoked internally by Catalina when the **–Ox** command line option is included in a Catalina command (where **x** is the optimization level – currently **1**, **2, 3, 4** or **5** are supported).

For example:

```
catalina -lc hello_world.c -O4
```

Three levels of code optimization are currently supported:

- `-O1` : basic optimization (single pass optimization)
- `-O2` : intensive optimization (performs a second optimization pass)
- `-O3` : same as -O2 but with automatic inlining
- `-O4` : same as -O3 but with automatic elimination of unused code
- `-O5` : same as -O3 but with load optimization

Note that unlike many other Catalina command line options, there can be no space between the **–O** and the level (**1**, **2**, **3**, **4** or **5**).

The single optimization pass (`-O1`) does most of the work, and typically reduces the size of code segments by 10%. The increase in overall program performance achieved typically ranges from 5% to 10% depending on the program itself (the larger savings are typical for programs that use the XMM memory model).

The second optimizer pass (`-O2`) can save an additional few percent in both space and execution time in larger programs.

The third option (`-O3`) searches for functions where inlining the body of the function would save the overhead of actually performing the function call - provided the inlining does not make the code size any larger (this optimization can make a very large difference in programs that are constructed from many small functions).

The fourth option (`-O4`) removes functions from the code segment – i.e. functions that are not referenced by the main program or any function referenced – directly or indirectly - by the main program.

The fifth option (`-O5`) optimizes load operations that can be performed in less space – e.g. loads that can be done in immediate mode rather than from Hub RAM.

Note that the optimizer works on ALL the C code in the program - including library functions (i.e. you don't need to maintain two versions of either the standard C libraries or your own libraries - one compiled with the optimizer and one without).

Using the Optimizer with BlackBox and BlackCat

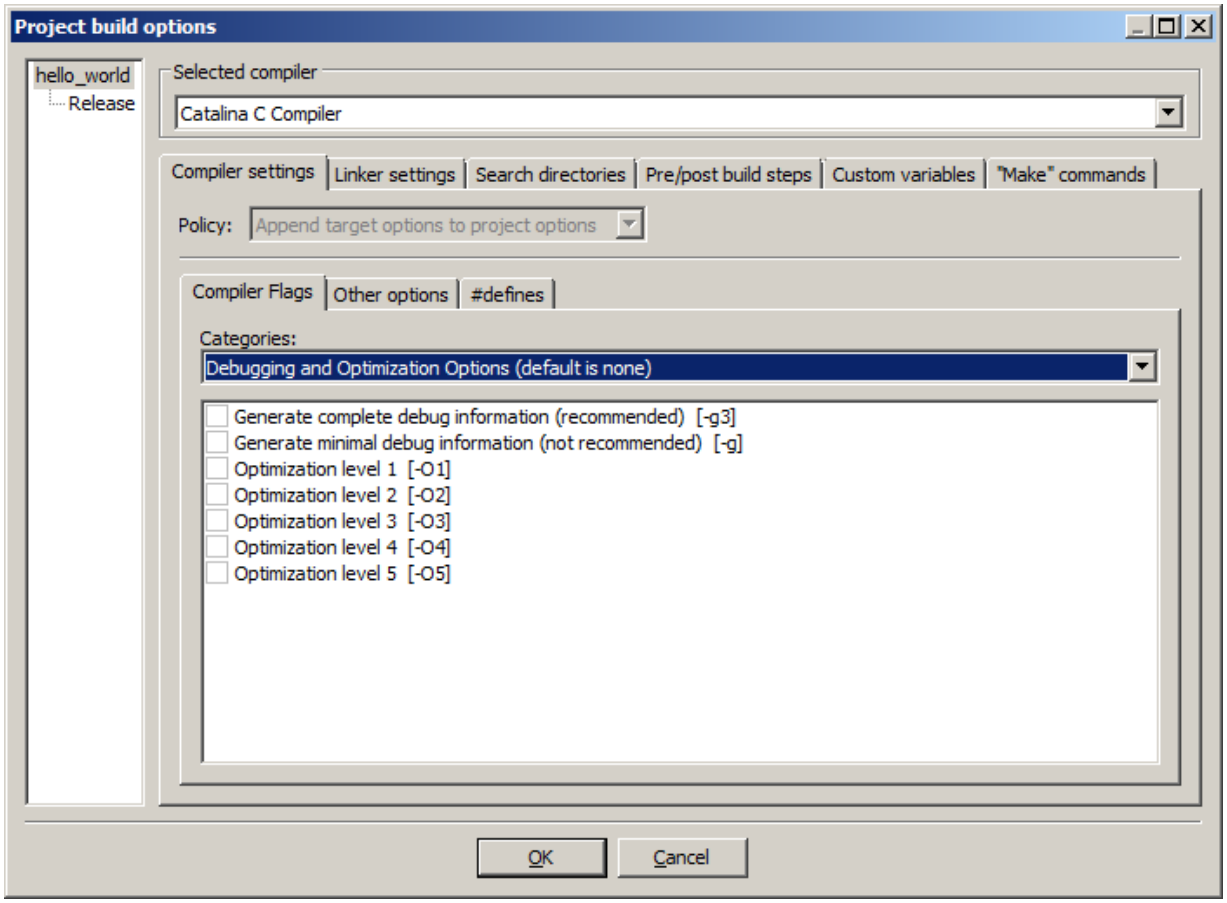
Optimized code can still be used with either the BlackBox and BlackCat debuggers, except that inlining should probably not be used if you want to debug functions that may have been inlined – most debugger operations will work correctly on optimized code (e.g. break and single step), but those operations that depend on the debugger identifying the return address by reading the stack frame may (e.g. step out, step up) will not work correctly on inlined functions.

Using the Optimizer with Code::Blocks

The Optimizer can be used with Code::Blocks simply by selecting the desired optimization command line option (e.g. **-O3**) as an additional linker option in the appropriate Code::Blocks project.

In Code::Blocks, select the menu command **Project->Build Options**, then in the dialog box that appears, select the top level project in the left hand navigation panel, select the **Compiler Flags** tab, then select the **Debugging and Optimization Options** category. Select **-O1**, **-O2**, **-O3**, **-O4** or **-O5**, as shown below:

Then build the project as normal.



Optimizer Options

Although it is not intended to be invoked manually, if the **–v** (verbose) option is used in a Catalina compilation the optimizer can be seen being invoked with various options. So here is a list of the options supported by the program.

```
-? or -h  print a helpful message (and exit)
-b        binary output (default)
-d        generate listing
-e        eeprom output;
-ilevel   set information level (note no space!)
-L path   path to libraries
-M size   memory size to use (default is 64k)
```

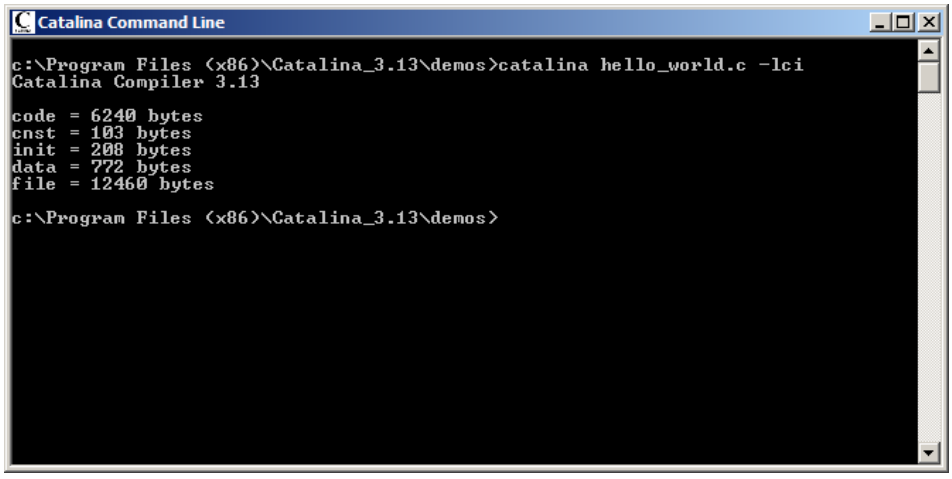
```
-o name    output optimized results to file 'name'
-Olevel   optimization level (-O1, -O2, -O3, -O4, -O5 - note no space!)
-u         do not remove intermediate output files
-p vers    Propeller version (1 or 2)
-v         verbose (output information messages)
-w         enable warnings
```

Optimizer Example

A normal Catalina compilation with no optimization, such as:

```
catalina -lc hello_world.c
```

might produce the following output:

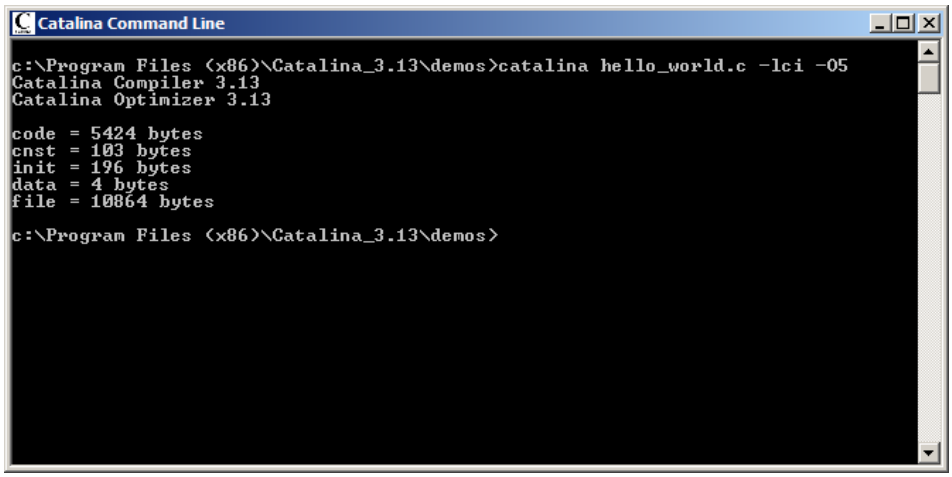


Note the code segment size for this program is 6240 bytes.

However, when optimization is included, such as:

```
catalina -lc hello_world.c -O5
```

the following output might be produced:



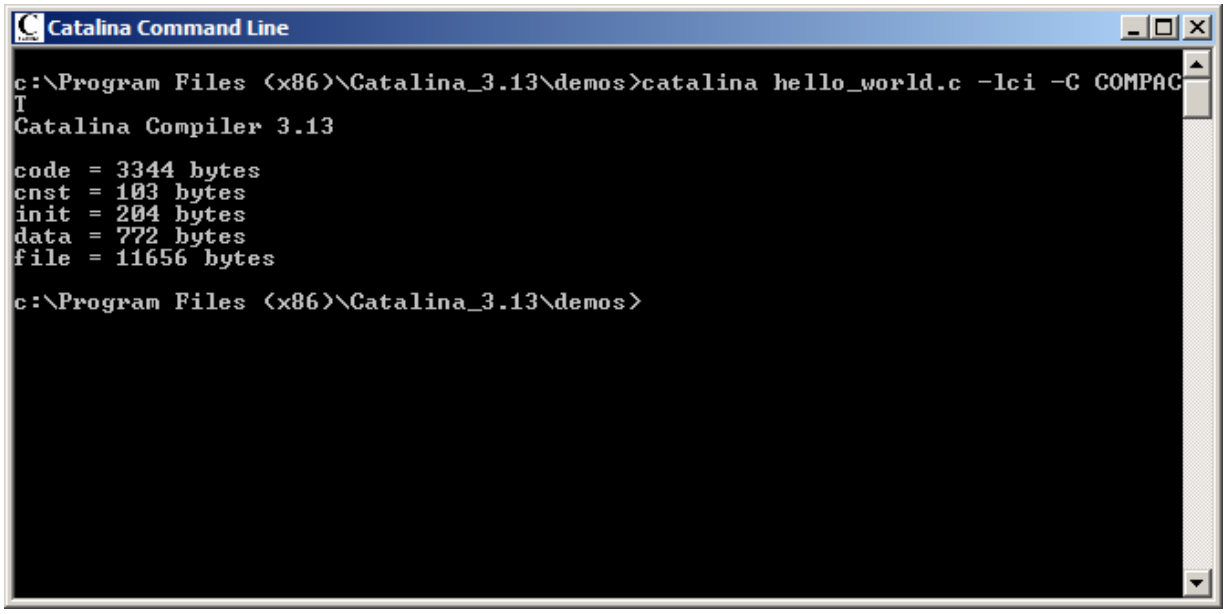
The code segment size is now 5424 bytes – a reduction in code size of 13%.

Note that when the Catalina Optimizer is used, the Catalina compilation process can take longer than normal.

The optimizer also works for CMM (Compact Memory Model) programs. For example, compiling the same program in compact mode, but with no optimization:

```
catalina -lc hello_world.c -C COMPACT
```

the following output might be produced:

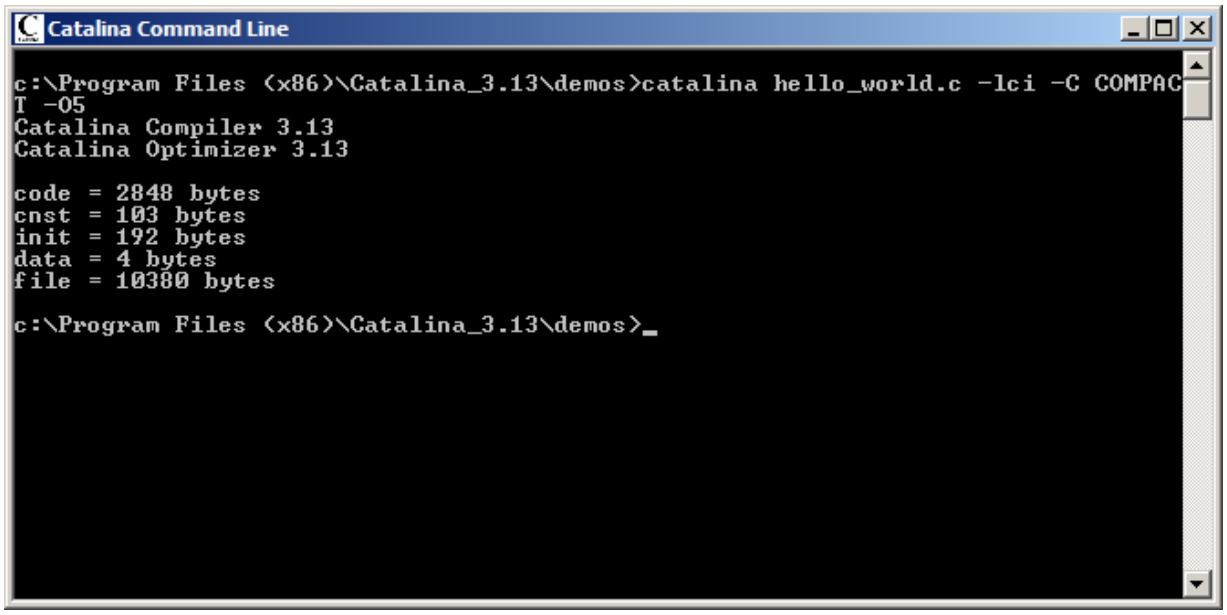


The code segment size is 3344 bytes.

When optimization is included:

```
catalina -lc hello_world.c -C COMPACT -O5
```

the following output might be produced:



The code segment size is now 2848 bytes – a reduction in code size of 15%.

Optimizer Phases

The Catalina Optimizer consists of the main program (**catoptimize** for LMM, XMM, NMM and **cmmoptimize** for CMM), which invokes up to fourteen separate optimization phases, represented by separate executable (**catopt_0** .. **catopt_13** for LMM, XMM, NMM and **cmmopt_0 .. cmmopt_13** for CMM). These phases cannot be used independently - each phase (except the first) typically requires the information generated by one or more of the preceding phases. The phases executed, and the order in which they are executed, depends on the optimization level specified.

The first phase is for optimizing loads, but (since it was added most recently) is invoked only when optimization level 5 is specified:

cxxopt_0 optimize the load instructions in the program to use smaller instructions where possible.

The next three phases are for inlining functions, and are invoked only when optimization level 3 (or graters) is specified:

cxxopt_1 identify all the potentially 'inlinable' function calls in the program.

cxxopt_2 identify all 'inlinable' functions, and generate an 'inline' version. To be inlinable, a function must:

not use the stack for arguments, *and*
not require any local variables, or use the frame pointer, *and*
not have its address taken elsewhere in the program, *and*
either be leaf function (i.e. one that makes no other calls),
or be called once only,
or be less than 6 longs in size.

cxxopt_3 replace all 'inlinable' function calls with an equivalent 'inlined' versions of the function.

The next three phases are for optimizing jumps. They are invoked when optimization level 1 (or greater) is specified:

cxxopt_4 identify all the absolute jumps in the program that can potentially be replaced by relative jumps. To do this, an intermediate version of the program must be compiled (to find out exactly how long each of the jumps are!)

cxxopt_5 identify all the relative jumps that are within the range -511 .. +511.

cxxopt_6 replace all the absolute jumps with relative jumps if they are within range.

The next three phases are for removing redundant instructions. They are invoked when when optimization level 1 (or greater) is specified:

cxxopt_7 identify all function calls in the program.

cxxopt_8 identify functions that don't spill arguments to stack (and therefore don't require the BC argument to be passed).

cxxopt_9 remove redundant assignments to BC for functions that don't use it.

cxxopt_10 remove redundant assignments from functions where the value assigned is not used again before the function returns.

The next two phases are for removing unreferenced code and data. They are invoked only when optimization level 4 (or greater) is specified:

cxxopt_11 identify all function calls and address loads in the program.

cxxopt_12 identify the functions used by each function, or whose address is taken.

cxxopt_13 remove functions not explicitly required.

Finally, if optimization level 2 (or greater) is specified, phases 4, 5 and 6 are then repeated – this is because the removal of redundant instructions sometimes allows additional absolute jumps to be turned into relative jumps (in the case of LMM, because the destination of the jump now falls into the range -511 .. +511).

Then the final version of the program is compiled.

Typical Optimizer Results

The amount of code that is saved by each optimization level can vary tremendously depending on the type of program. However, the following is quite typical:

```
Optimization Level 1: 10%
Optimization Level 2: 1-2%
Optimization Level 3: 2-3%
Optimization Level 4: 1-5%
Optimization Level 5: 1-5%
```

Optimization level 1 is fairly consistent at saving around 10% of code size purely due to the replacement of absolute jumps by relative jumps, and the elimination of redundant instructions.

Optimization level 2 may make no additional difference over and above optimization level 1 in small programs – in such programs it is typical that all possible jump optimizations have already been made in level 1. However, in larger programs it can often save an additional few percent of total code size.

Optimization level 3 may make no difference at all in programs that essentially consist of a single 'main' function, but in programs that consist of a relatively large number of small function calls, this optimization level can save up to 30% of the total code size, because of the overheads involved in making each function call.

Optimization level 4 will only make a significant difference if there are unused functions or variables in the program. However, this is often the case for C code that has been automatically generated, or in code that is still under development.

Optimization level 5 will make a significant difference for most programs.

Note that for optimization levels 1 to 3, it is only the *code segment size* that can be reduced – the size of any data segments (e.g. strings, initialized variables, jump tables etc) *cannot* be reduced unless there are unused functions or unused variables, and optimization level 4 is used..

Also, in a typical Catalina binary, quite a few kilobytes can taken up by the kernel and various drivers and plugins. However, this space is typically re-used as heap or stack at run time.

All this means is that you will not always see an apparent 15% reduction in the size of the final binary file even when the optimizer has removed 15% of the code. However, you will see significant performance improvements – usually on the same magnitude as the code savings.

For example, on the standard C **Dhrystone** integer benchmark program, the Catalina optimizer reduces the code size by around 15%, and increases the dhrystone performance by 8%. [1]

[1] Note that previous versions of the optimizer achieved better *apparent* improvements - this is because the Catalina compiler and kernels have themselves improved over the same time, leaving less scope for optimization to improve things further.

Optimizing hand-coded LMM PASM

The Catalina optimizer optimizes all code in the program, including library functions and hand-coded LMM PASM functions. However, when optimizing hand-coded LMM PASM, that PASM must be formatted so as to be recognizable by the Optimizer.

The following formatting rules must be obeyed:

1. No tabs embedded in the instructions;
2. All labels must start in column 1;
3. All comment markers **'** **{** or **}** must appear in column 1;
4. All opcodes must be in lower case. For example:

   mov *not* **MOV**

5. All special registers **BC**, **RI**, **FP** and **SP** must be in upper case (this does not apply to the general purpose registers r0 .. r23). For example:

   FP *or* **BC** *not* **fp** *or* **bc**

6. All LMM primitive names must be in upper case. For example:

   LODA *not* **loda**

7. Single spaces before the conditionals, between the conditionals and the opcode, between the opcode and the operands, and between the operands (with no space before the comma separating operands). For example:

```
if_z mov FP, #1
^    ^   ^   ^   ← single spaces should appear here
```

8. No comments or blank lines between the parts of an LMM primitive. For example:

```
jmp #LODA
                ← no blank line or comment should appear here
long @my_variable
```

If in doubt, follow the code format used by the generated code or the hand-coded library functions as closely as possible.

For example, here is an actual (but quite trivial) Catalina library function that will be recognized as being able to be 'inlined' by the Catalina Optimizer:

```
C__ina
 mov r0, INA
 jmp #RETN
```

If the **–O3** option is selected when optimizing, all calls to the library function **_ina()** will be replaced by the single inline instruction:

```
mov r0, INA
```

This not only saves one instruction in the body of the function itself, it saves up to 3 instructions for each invocation of the function.

The Current Optimizer Release

What's new in this release?

- Nothing.

What's due in the next release?

- To be determined!

Reporting Bugs

Please report all Optimizer bugs to ross@thevastydeep.com.

Where possible, please include a *brief* example program that demonstrates the problem. Ideally, this will be in the form of a program that works correctly without optimization, but fails to compile or run correctly when optimization is used.